

CI/CD Security Gate Integration

Software Composition Analysis (SCA) using Snyk

Project: webapp-cicd-pipeline | Tool: Snyk CLI | Orchestrator: Jenkins Declarative Pipeline

1. Objective

This document records the steps followed to integrate Software Composition Analysis (SCA) into the existing Jenkins CI/CD pipeline using Snyk. SCA scans project dependencies (e.g. Maven libraries) for known vulnerabilities (CVEs) and fails the build if high or critical severity issues are found, before the artifact reaches the Build and Deploy stages.

The pipeline stage order after this integration is:

Initialize → Secret Scan (TruffleHog) → SCA Scan (Snyk) → SAST Scan (Semgrep) → Build → Deploy To Tomcat

2. Prerequisites

- A Snyk account (free tier is sufficient) at snyk.io
- Jenkins server with administrator access to add credentials
- SSH access to the Jenkins build agent (Ubuntu) to install the Snyk CLI
- A Maven-based project already connected to the pipeline (pom.xml present)

3. Step 1 — Install the Snyk CLI on the Jenkins Agent

The Snyk CLI binary was downloaded directly onto the Ubuntu build agent, made executable, and moved into a directory already present on the system PATH so the Jenkinsfile can invoke it.

```
ubuntu@ip-172-31-8-38:~$ curl -Lo snyk https://static.snyk.io/cli/latest/snyk-linux
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           %             Dload  Upload  Total   Spent    Left   Speed
100 183.3M  0 183.3M  0     0 83.29M      0  --:--:--  0:00:02  74.46M

ubuntu@ip-172-31-8-38:~$ chmod +x snyk
ubuntu@ip-172-31-8-38:~$ sudo mv snyk /usr/local/bin/
```

Verification: the CLI is confirmed to be on the PATH and reachable from within the pipeline:

```
which snyk
snyk --version
```

In the Jenkinsfile, an environment variable `SNYK_PATH = "/usr/local/bin"` is defined and appended to PATH inside every shell step of the SCA stage, so the pipeline does not depend on the default PATH of the Jenkins agent's shell.

4. Step 2 — Generate a Snyk API Token

Snyk authenticates CLI requests using a Personal Access Token (PAT). This token is generated once from the Snyk web console and is what Jenkins will use to run scans non-interactively.

1. Log in to app.snyk.io and open Account Settings.
2. Navigate to Personal Access Tokens.
3. Provide a Name for the token, set an Expiry, and click Generate new token.
4. Copy the generated token immediately — it is only shown once.

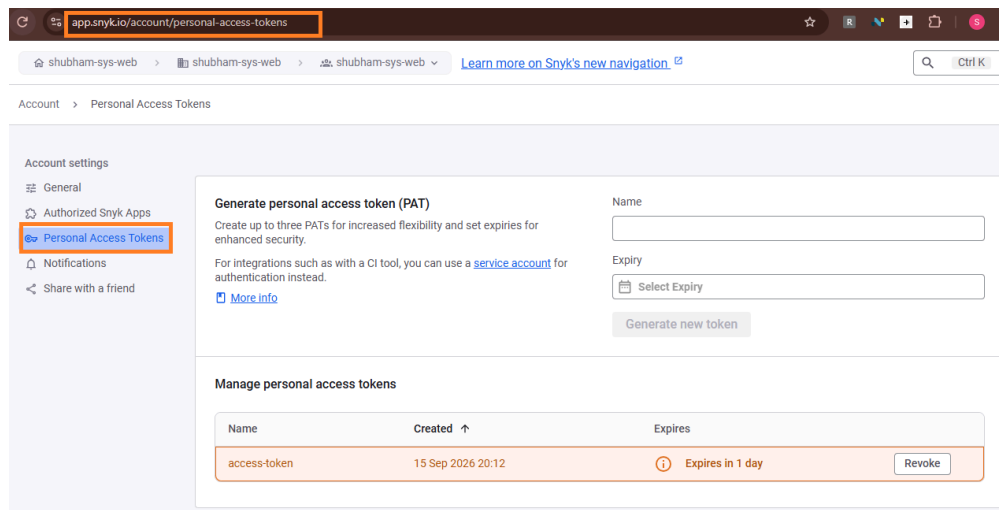


Figure 1 — Generating a Personal Access Token from the Snyk console (Account → Personal Access Tokens).

5. Step 3 — Store the Token as a Jenkins Credential

The token must never be hardcoded in the Jenkinsfile. It is stored in the Jenkins credential store and injected at runtime using `withCredentials()`.

1. In Jenkins, go to Manage Jenkins → Credentials → System → Global credentials (unrestricted).
2. Click Add Credentials.
3. Select Secret text as the credential Kind.

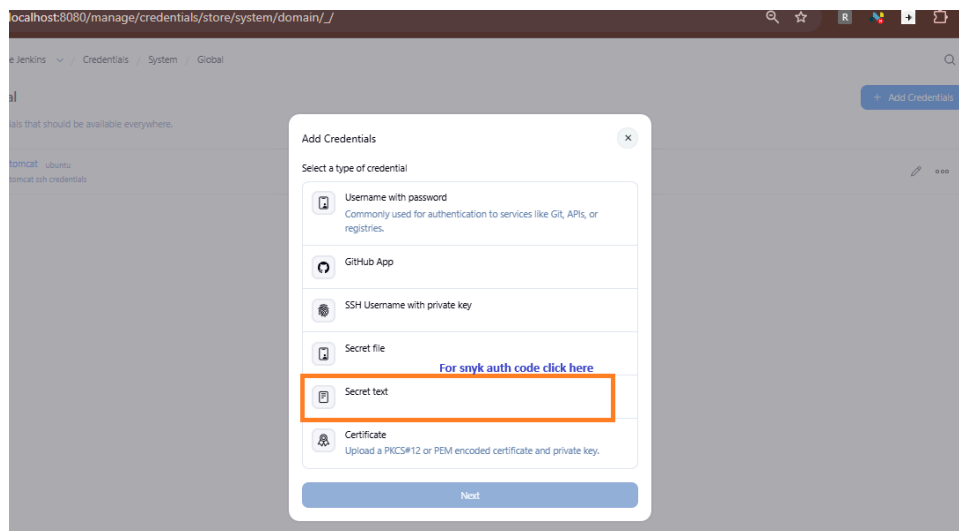


Figure 2 — Selecting "Secret text" as the credential type in Jenkins.

4. Paste the Snyk token generated in Step 2 into the Secret field.
5. Set the ID field to exactly: `snyk-token` (this must match the `credentialsId` used in the Jenkinsfile — it is case-sensitive).
6. Add a short Description (e.g. "Snyk API Token") and click Create.

Figure 3 — Adding the Snyk token as a Secret text credential with ID "snyk-token".

6. Step 4 — Add the SCA Stage to the Jenkinsfile

A new stage, SCA Scan - Snyk, was added to the declarative pipeline. It authenticates using the stored credential, runs a full dependency scan to produce a JSON report for the build artifacts, and then runs a second, gating scan that fails the build if any high or critical severity vulnerability is found.

```
stage('SCA Scan - Snyk') {
  steps {
    withCredentials([string(credentialsId: 'snyk-token', variable: 'SNYK_TOKEN')]) {
      script {
        sh '''
          export PATH=$PATH:${SNYK_PATH}
          which snyk
          snyk --version
        '''

        // Full JSON report (non-blocking so we always get the report file)
        sh '''
          export PATH=$PATH:${SNYK_PATH}
          snyk test --json > snyk-report.json || true
        '''
        sh 'cat snyk-report.json'

        archiveArtifacts artifacts: 'snyk-report.json', allowEmptyArchive: true

        // Gate the pipeline on high/critical severity
        def snykExitCode = sh(
          script: '''
            export PATH=$PATH:${SNYK_PATH}
            snyk test --severity-threshold=high
          ''',
          returnStatus: true
        )

        if (snykExitCode != 0) {
```

```
        error "Pipeline stopped: high/critical severity vulnerabilities found."
    } else {
        echo "No high/critical SCA vulnerabilities found."
    }
}
}
```

Note: SNYK_TOKEN is picked up automatically by the Snyk CLI from the environment — no separate snyk auth step is required once the credential is bound with withCredentials().

7. Step 5 — Run the Pipeline and Verify

With the credential in place, the pipeline was re-triggered. The build progressed past Initialize and Secret Scan - TruffleHog into the new SCA Scan - Snyk stage, which produced a live JSON vulnerability report in the console output.

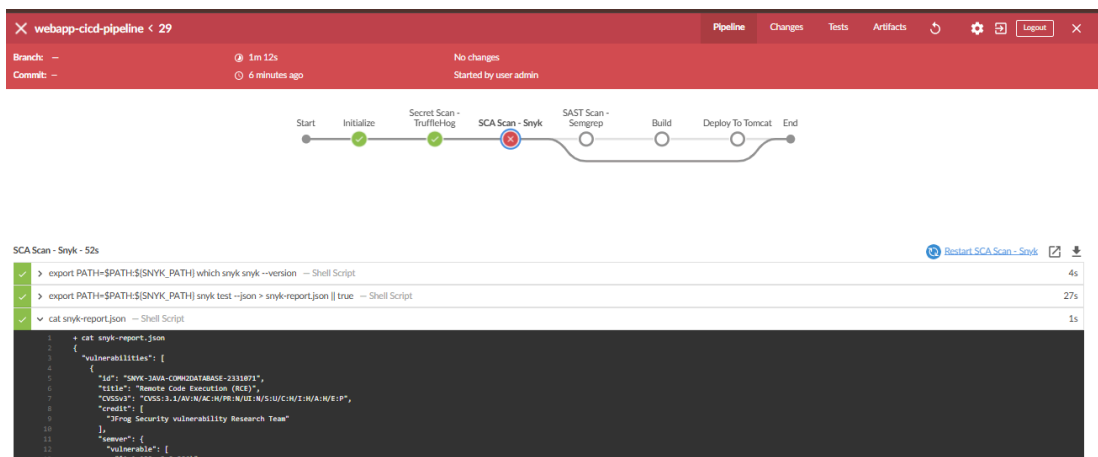


Figure 4 — Jenkins Blue Ocean view: pipeline reaches the SCA Scan - Snyk stage and streams snyk-report.json to the console.

Findings observed in this run (from snyk-report.json), used as proof that the gate is functioning correctly:

- `com.h2database:h2 @ 1.4.197` — 2 Remote Code Execution (RCE) vulnerabilities, one rated CVSS 9.8 (Critical).
- `org.apache.logging.log4j:log4j-core @ 2.14.1` — CVE-2021-44228 ("Log4Shell"), a critical Remote Code Execution vulnerability.

Because the stage is configured with `--severity-threshold=high`, the build correctly failed (FAILURE) at this stage instead of proceeding to Build and Deploy — this is the intended behaviour of a DevSecOps quality gate, not a pipeline defect.

8. Remediation Applied

1. Upgrade `com.h2database:h2` in `pom.xml` from 1.4.197 to 2.1.210 or later.
2. Upgrade `org.apache.logging.log4j:log4j-core` in `pom.xml` from 2.14.1 to 2.25.5 or later (resolves Log4Shell and related CVEs).
3. Run `mvn clean package` locally to confirm the project still builds against the updated dependency versions.
4. Commit and push the changes so the pipeline re-runs; the SCA Scan - Snyk stage is expected to pass once vulnerable versions are removed.

9. Summary

The Snyk CLI was installed on the Jenkins agent, authenticated using a Personal Access Token stored as a Jenkins "Secret text" credential (ID: snyk-token), and wired into the pipeline as a dedicated SCA Scan - Snyk stage placed after the Secret Scan and before the SAST Scan. The stage archives a full JSON report as a build artifact and fails the pipeline on any high or critical severity dependency vulnerability, giving the pipeline three independent security gates — secrets, dependencies (SCA), and source code (SAST) — before an artifact can be built and deployed.